

AGENDA

- We will discuss:
 - Polynomial Regression for Classification
 - Logistic Regression
 - Evaluation Metrics for Classification

At the end of this lecture, you should be able to:

- ✓ Explain the underlying principles of logistic regression
- ✓ Apply the two algorithms for classification
- ✓ Evaluate the classification performance

Recap: Classification Task

Input: $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)$

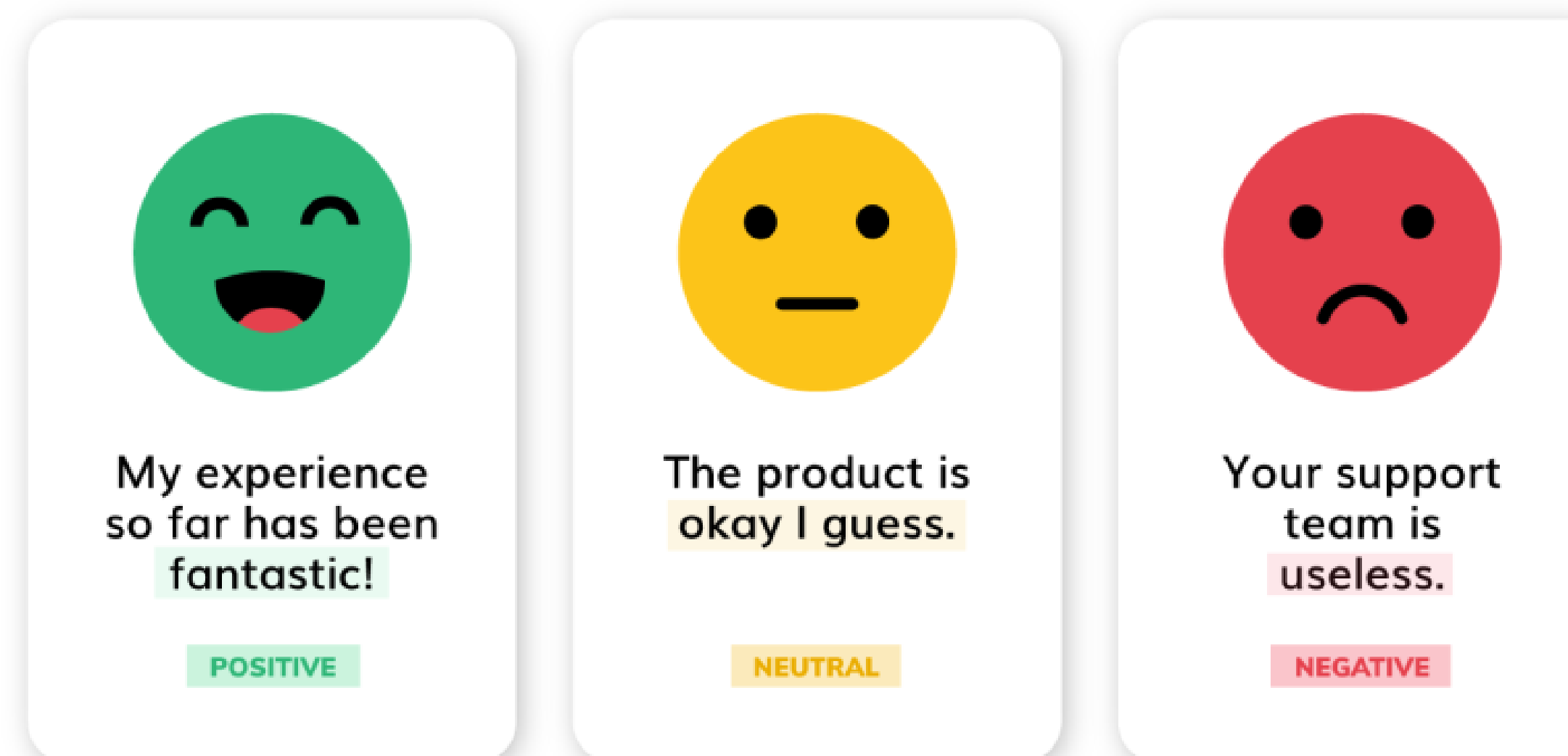
Output: $f(\mathbf{x})$ that predict **categorical** y given $\mathbf{x}$

Applications:

Spam Detection



Sentiment Analysis



That is life	English
C'est la vie	Portuguese
Yahee jeevan hai	Hindi
Bubomi obo	isiXhosa
それが人生です	Japanese

Image Classification

airplane

automobile

bird

cat

deer

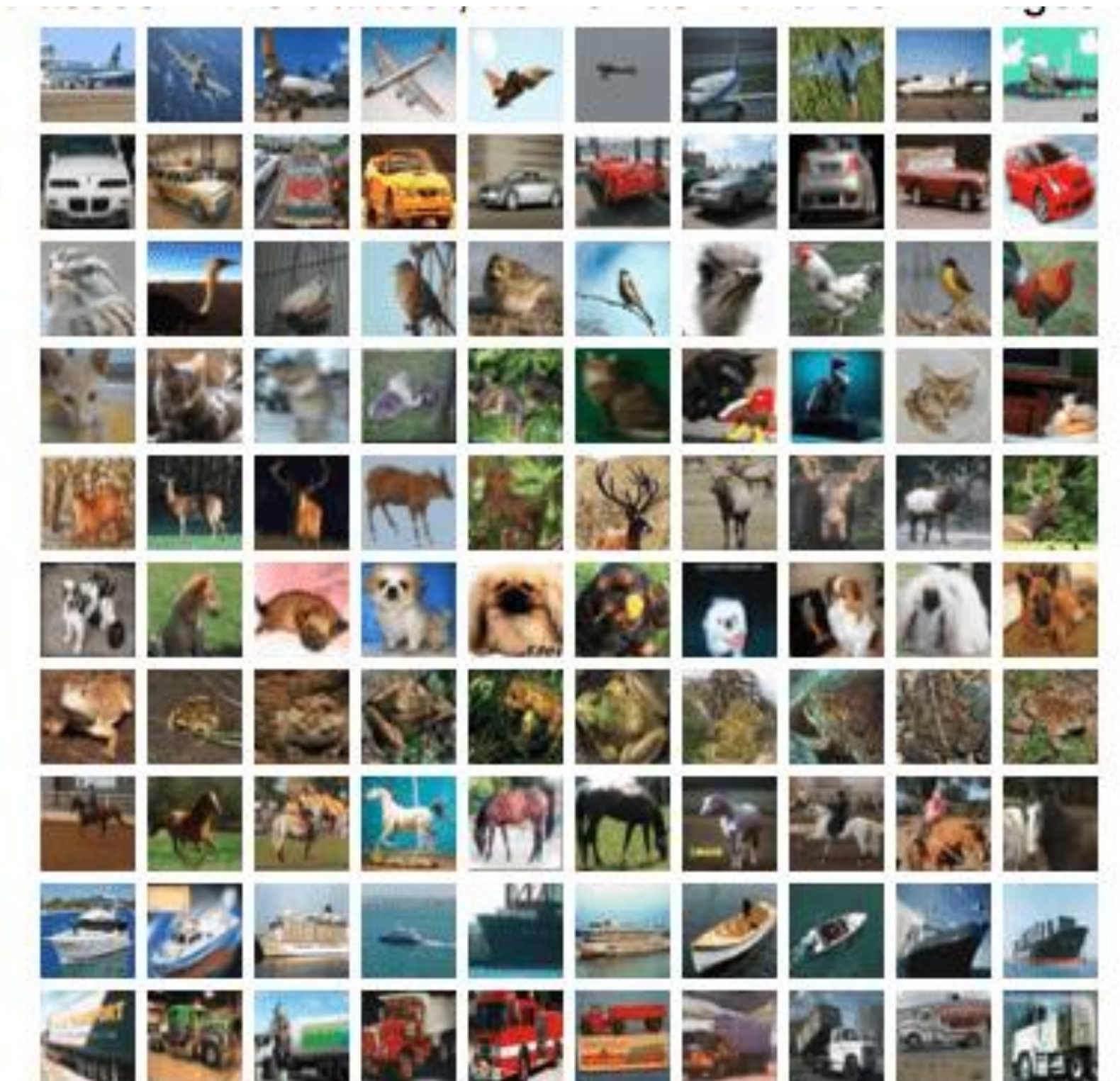
dog

frog

horse

ship

truck



Language
Identification

Polynomial Regression for Classification?

➤ Binary Classification

Input: $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)$

Output: $f_{\mathbf{w}}(\mathbf{x}) = f_{\mathbf{w}}(\mathbf{p}(\mathbf{x})) = \text{sign}(\mathbf{p}^T \mathbf{w})$

Threshold function

Negative
class

Positive
class

$$y_i \in \{-1, +1\}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

If $\mathbf{P}^T \mathbf{P}$ is invertible, then

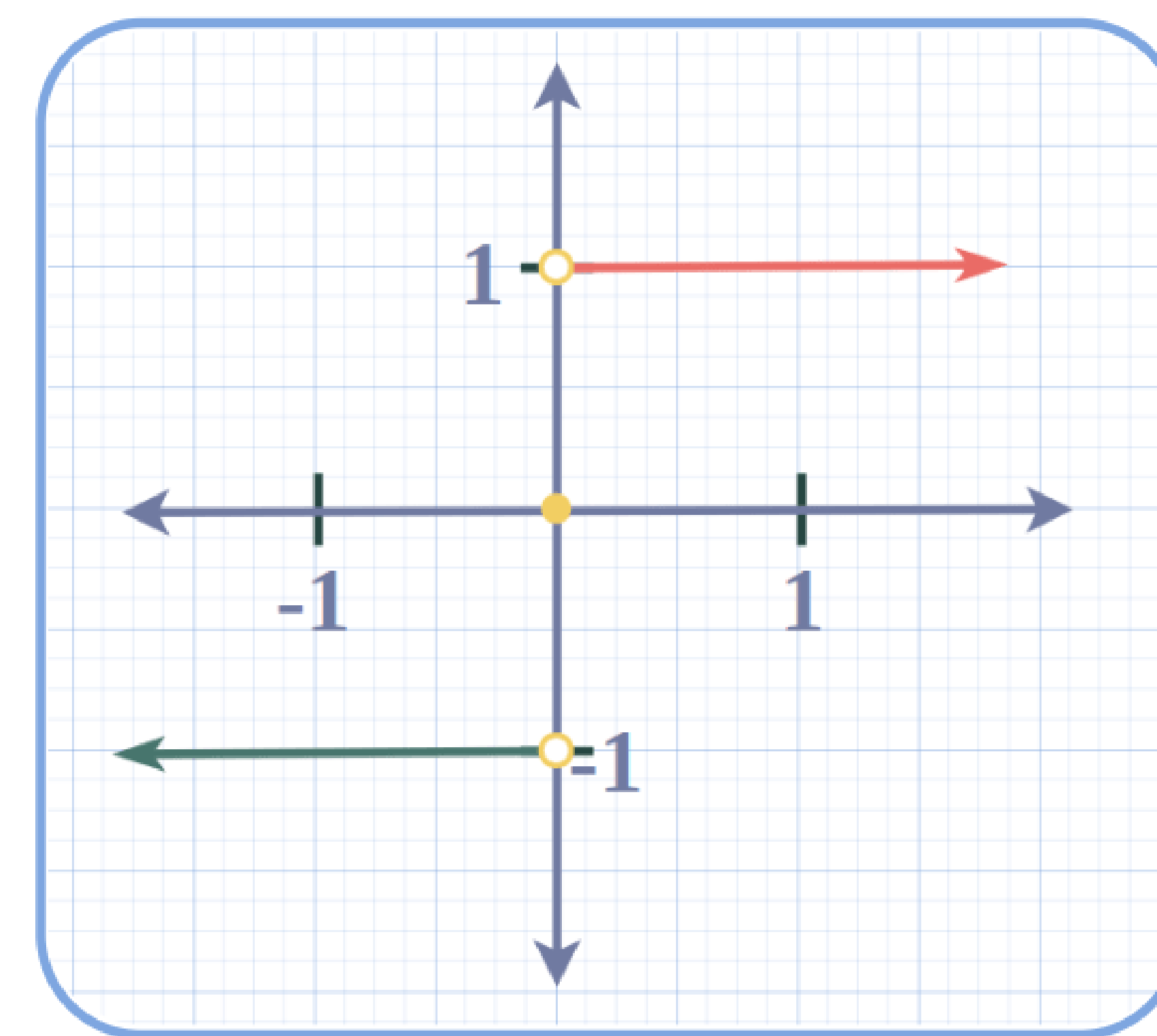
-Learning: $\mathbf{w}^* = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{y}$

-Prediction: $\mathbf{f}_{\mathbf{w}^*}(\mathbf{P}(\mathbf{X}_t)) = \text{sign}(\mathbf{P}_t \mathbf{w}^*)$

$$(\mathbf{p}_i^t)^T \mathbf{w}^* = 0?$$

$$= \begin{bmatrix} \text{sign}((\mathbf{p}_1^t)^T \mathbf{w}^*) \\ \vdots \\ \text{sign}((\mathbf{p}_m^t)^T \mathbf{w}^*) \end{bmatrix}$$

Graph of Signum Function



$$\text{sign}(z) = \begin{cases} -1, & z < 0 \\ 0, & z = 0 \\ +1, & z > 0 \end{cases}$$

Polynomial Regression for Classification?

➤ Binary Classification

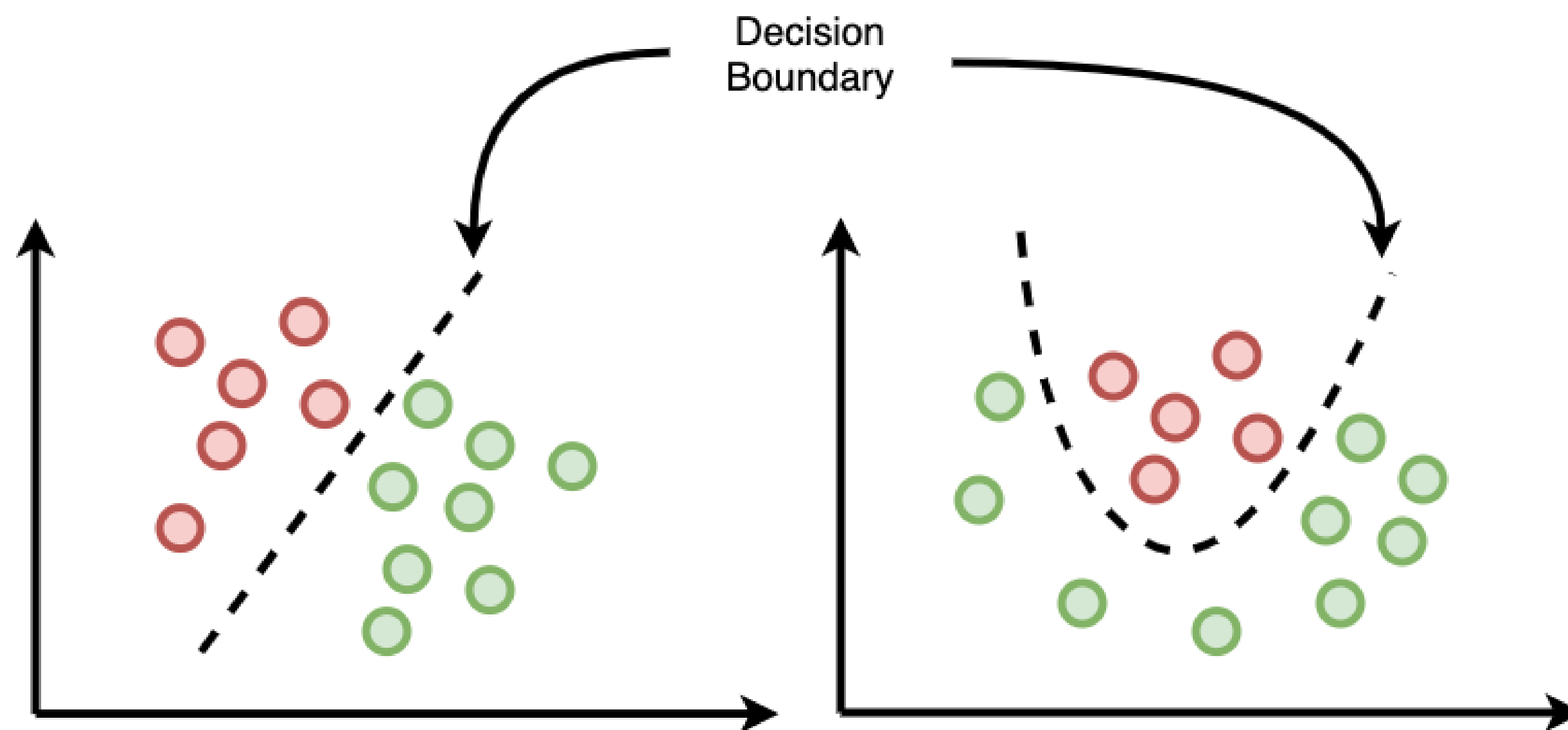
Input: $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)$

Output: $f_{\mathbf{w}}(\mathbf{x}) = f_{\mathbf{w}}(\mathbf{p}(\mathbf{x})) = \text{sign}(\mathbf{p}^T \mathbf{w})$

Negative class Positive class
 $y_i \in \{-1, +1\}$ $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$

▪ Decision Boundary: $\mathbf{p}^T \mathbf{w} = 0$

(a line or a surface separates the two classes)



$$\text{sign}(z) = \begin{cases} -1, & z < 0 \\ 0, & z = 0 \\ +1, & z > 0 \end{cases}$$

What if threshold function

$$th(z) = \begin{cases} -1, & z < 0.3 \\ 0, & z = 0.3 \\ +1, & z > 0.3 \end{cases}$$

Polynomial Regression for Classification?

➤ Multi-category Classification

Input: $(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_N, \mathbf{y}_N)$

$\mathbf{y}_i$ is K -Dimensional one-hot encoded vector

e.g., $K=3$ categories: dog, cat, rabbit.

↓ One-hot encoding

dog=[1,0,0], cat=[0,1,0], rabbit=[0,0,1]

Output: $f_{\mathbf{W}}(\mathbf{x}) = f_{\mathbf{W}}(\mathbf{p}(\mathbf{x})) = \underset{k=1, \dots, K}{\operatorname{argmax}}(\mathbf{p}^T \mathbf{W})$

How does decision boundary look like?

If $\mathbf{P}^T \mathbf{P}$ is invertible, then

-Learning: $\mathbf{W}^* = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{Y}$

-Prediction: $\mathbf{f}_{\mathbf{W}^*}(\mathbf{P}(\mathbf{X}_t)) = \begin{bmatrix} \underset{k=1, \dots, K}{\operatorname{argmax}}((\mathbf{p}_1^t)^T \mathbf{W}^*) \\ \vdots \\ \underset{k=1, \dots, K}{\operatorname{argmax}}((\mathbf{p}_m^t)^T \mathbf{W}^*) \end{bmatrix}$

Polynomial Regression for Classification?

➤ Multi-category Classification

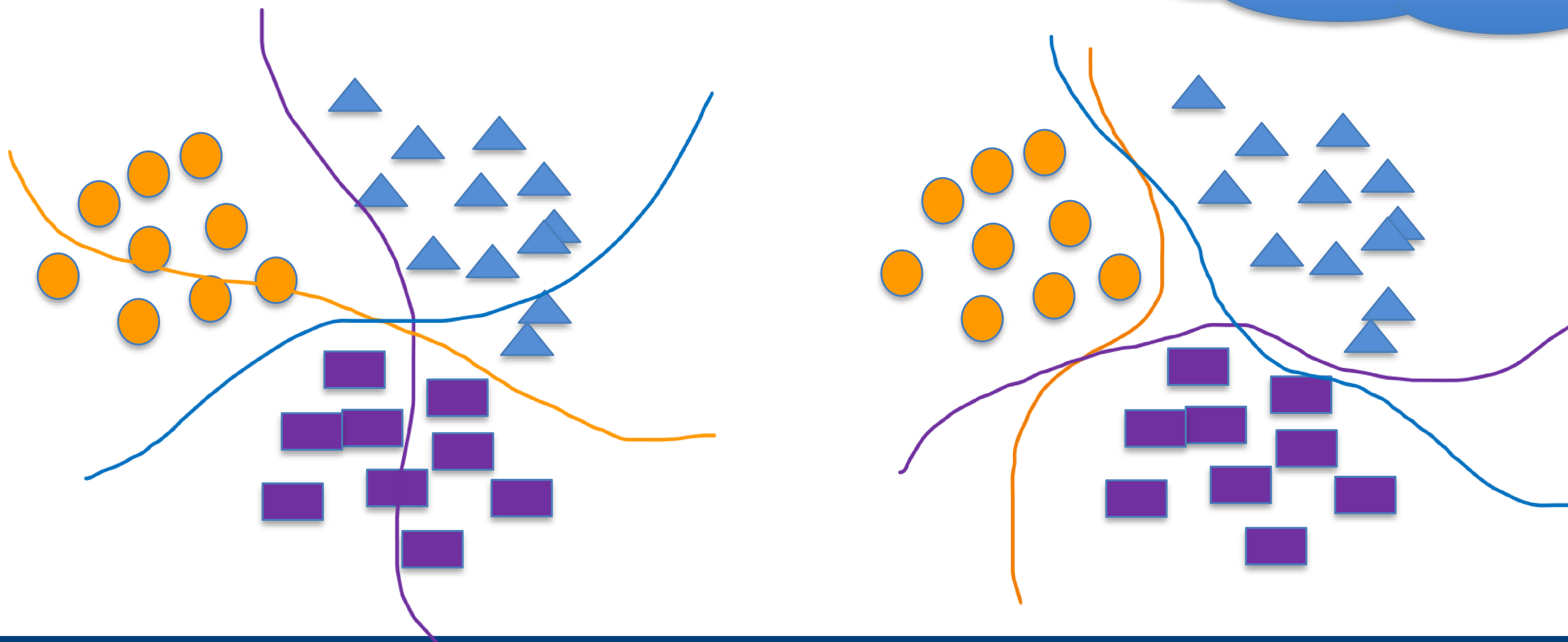
Input: $(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_N, \mathbf{y}_N)$

$\mathbf{y}_i$ is K -Dimensional one-hot encoded vector

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Output: $f_{\mathbf{W}}(\mathbf{x}) = f_{\mathbf{W}}(\mathbf{p}(\mathbf{x})) = \underset{k=1, \dots, K}{\operatorname{argmax}} (\mathbf{p}^T \mathbf{W})$

How does decision boundary look like?



Polynomial Regression for Classification?

➤ Multi-category Classification

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class3}), (0.8, \text{class2})\}$, predict the class label for the test dataset $\{-0.1, 0.4\}$ using 1st order polynomial regression.

Step 0: Polynomial Transformation ($\mathbf{X} \rightarrow \mathbf{P}$) and One-hot Encoding

1st order polynomial model:

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 = \begin{bmatrix} 1 & x_1 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

$$\mathbf{P} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 0.5 \\ 1 & 0.3 \\ 1 & 0.8 \end{bmatrix}$$

Form
target
matrix

$$\begin{aligned} \text{class1: } & [1, 0, 0] \\ \text{class2: } & [0, 1, 0] \\ \text{class3: } & [0, 0, 1] \end{aligned}$$

$$\mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Polynomial Regression for Classification?

➤ Multi-category Classification

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class3}), (0.8, \text{class2})\}$, predict the class label for the test dataset $\{-0.1, 0.4\}$ using 1st order polynomial regression.

Step 1: Learning (work out $\mathbf{W}^*$)

Since $\mathbf{P}^T \mathbf{P}$ is invertible, $\mathbf{W}^* = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{Y} = \begin{bmatrix} 0.4780 & 0.3333 & 0.1887 \\ -0.6499 & 0.5556 & 0.0943 \end{bmatrix}$

Step 2: Prediction

$$\mathbf{P}_t = \begin{bmatrix} 1 & x_{1,1}^t \\ 1 & x_{2,1}^t \end{bmatrix} = \begin{bmatrix} 1 & -0.1 \\ 1 & 0.4 \end{bmatrix}$$

$$\mathbf{Y}_t^* = \mathbf{P}_t \mathbf{W}^* = \begin{bmatrix} 1 & -0.1 \\ 1 & 0.4 \end{bmatrix} \begin{bmatrix} 0.4780 & 0.3333 & 0.1887 \\ -0.6499 & 0.5556 & 0.0943 \end{bmatrix}$$

$$= \begin{bmatrix} 0.5430 & 0.2778 & 0.1792 \\ 0.2180 & 0.5556 & 0.2264 \end{bmatrix} \xrightarrow{\text{argmax}}$$

Handwritten notes: "class1" with an arrow pointing to the first row of the matrix, and "class2" with an arrow pointing to the second row of the matrix. A blue arrow points from the matrix to the handwritten "class2".

Polynomial Regression for Classification?

➤ Not Recommended for Classification

1. Hard Threshold Function, e.g., “sign()”

Completely confident prediction

$$\mathbf{P}_t \mathbf{w}^* = \begin{bmatrix} -5 \\ 10 \\ 0.01 \end{bmatrix} \xrightarrow{\text{sign()}} \hat{\mathbf{y}}_t = \begin{bmatrix} -1 \\ 1 \\ \textcircled{1} \end{bmatrix}$$

Near decision boundary

2. $\mathbf{p}^T \mathbf{w}$ is a number between $-\infty$ and $+\infty$. *Need uncertainty method to avoid such data*

The regression model output values are not interpretable

$$\mathbf{P}_t \mathbf{W}^* = \begin{bmatrix} -4 & \textcircled{4.2} & 2.5 \\ 1 & -0.8 & 0.2 \end{bmatrix}$$

(With **soft** threshold function)

Logistic Regression

-Used for classification task, not regression task

Binomial Logistic Regression

➤ Model (Binary)

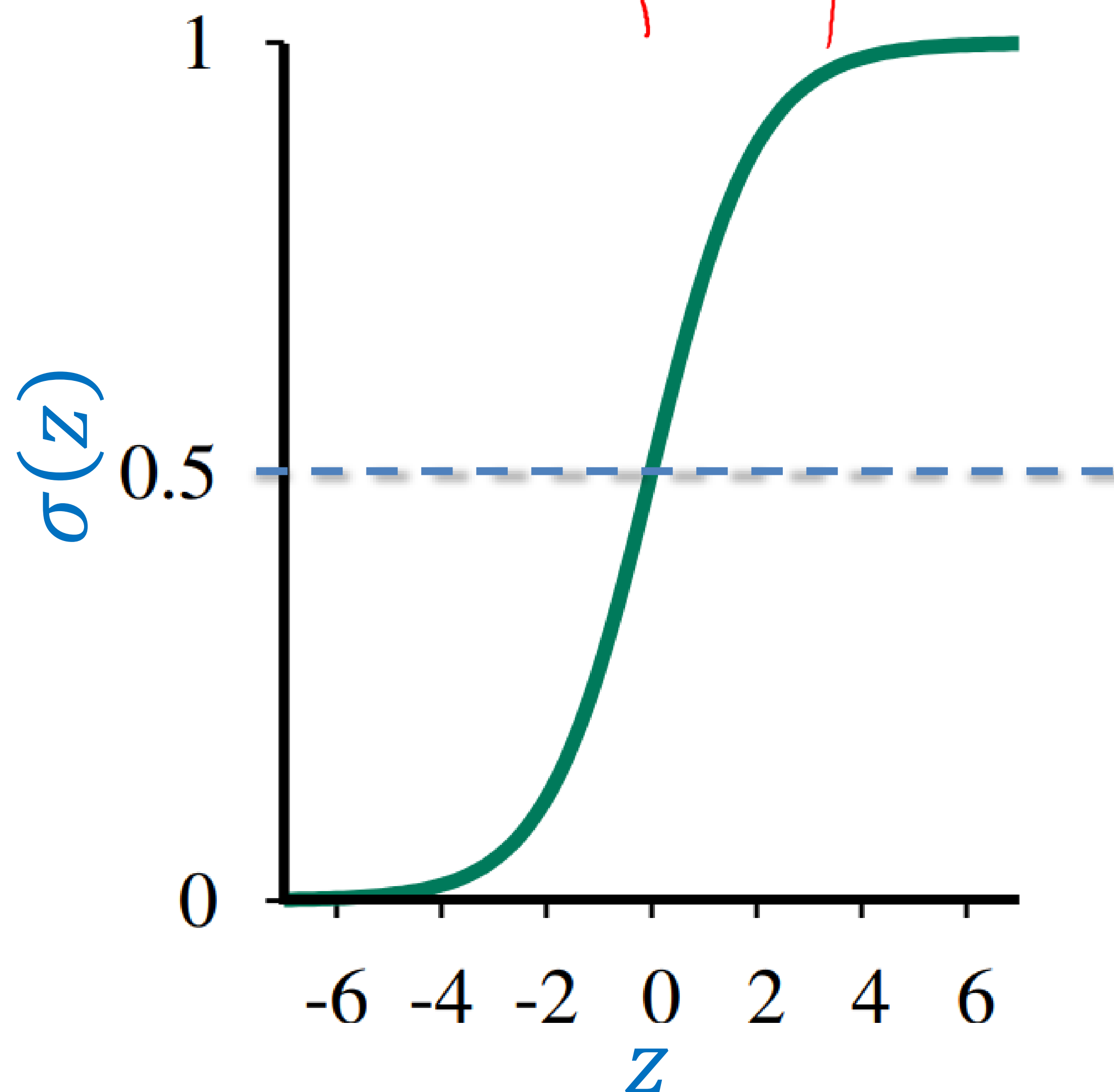
Input: $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N), y_i \in \{0, 1\}$ $\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}$

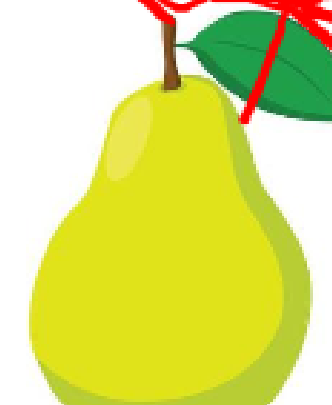
Output: $f_{\mathbf{w}}(\mathbf{x}) = \sigma(\mathbf{x}^T \mathbf{w})$, where $\sigma(z) = \frac{1}{1+e^{-z}}$

Negative class Positive class

Sigmoid (logistic) function

- Value range: (0,1) → Probability



e.g.,  vs  *Likely an apple*

Label (y_i): 0 1

If $f_{\mathbf{w}}(\mathbf{x}_{new}) = 0.2$, $P(? | ?) = 0.2$
 If $f_{\mathbf{w}}(\mathbf{x}_{new}) = 0.6$, $P(? | ?) = 0.6$

Binomial Logistic Regression

➤ Learning (Binary)

Find the optimal values for $\mathbf{w}^*$ which minimizes:

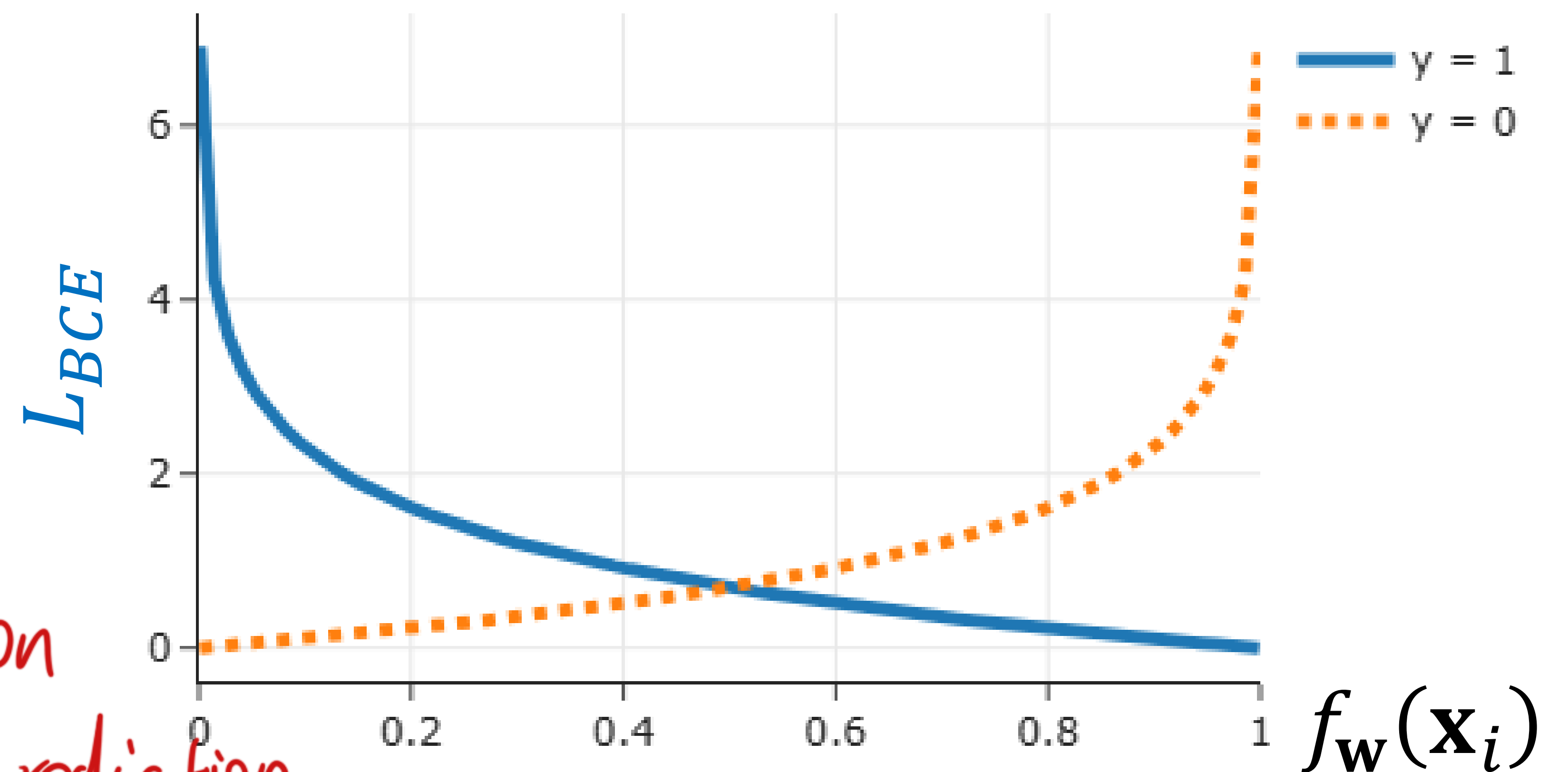
$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \text{Loss}(f_{\mathbf{w}}(\mathbf{x}_i), y_i)$$

where $\text{Loss}(f_{\mathbf{w}}(\mathbf{x}_i), y_i) = -y_i \ln(f_{\mathbf{w}}(\mathbf{x}_i)) - (1 - y_i) \ln(1 - f_{\mathbf{w}}(\mathbf{x}_i))$

Binary Cross-Entropy Loss

$$L_{BCE}(f_{\mathbf{w}}(\mathbf{x}_i), y_i) =$$

$$\begin{cases} -\ln(1 - f_{\mathbf{w}}(\mathbf{x}_i)), & \text{if } y_i = 0 \\ -\ln(f_{\mathbf{w}}(\mathbf{x}_i)), & \text{if } y_i = 1 \end{cases}$$



The penalise for far prediction is stronger than correct prediction.

Binomial Logistic Regression

➤ Learning (Binary)

$$\operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^N L_{BCE}(f_{\mathbf{w}}(\mathbf{x}_i), y_i)$$

$$\operatorname{argmax}_{\mathbf{w}} F = \operatorname{argmin}_{\mathbf{w}} -F$$



$$\ln \left(\prod_{i=1}^N a_i \right) = \sum_{i=1}^N \ln(a_i)$$

Maximum
Likelihood
Estimation

$$\operatorname{argmax}_{\mathbf{w}} \ln \left(\prod_{i=1}^N f_{\mathbf{w}}(\mathbf{x}_i)^{y_i} (1 - f_{\mathbf{w}}(\mathbf{x}_i))^{(1-y_i)} \right)$$

Likelihood of
correct
assumption

Product of all

(Assumption: data samples
are **independent**.)

if $y_i = 0$,
 $1 - f_{\mathbf{w}}(\mathbf{x}_i)$
 $P(y_i = 0 \mid \mathbf{x}_i)$

if $y_i = 1$,
 $f_{\mathbf{w}}(\mathbf{x}_i)$
 $P(y_i = 1 \mid \mathbf{x}_i)$

(Assumption: data samples
are **identically distributed**.)

Binomial Logistic Regression

➤ Learning (Binary)

$$\operatorname{argmin}_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N L_{BCE}(f_{\mathbf{w}}(\mathbf{x}_i), y_i)$$

no closed-form solution!

Recall: **Gradient Descent Algorithm!**

Goal: $\operatorname{argmin}_{\mathbf{w}} J(\mathbf{w})$

Initialize $\mathbf{w}_0$ and learning rate η
while true do

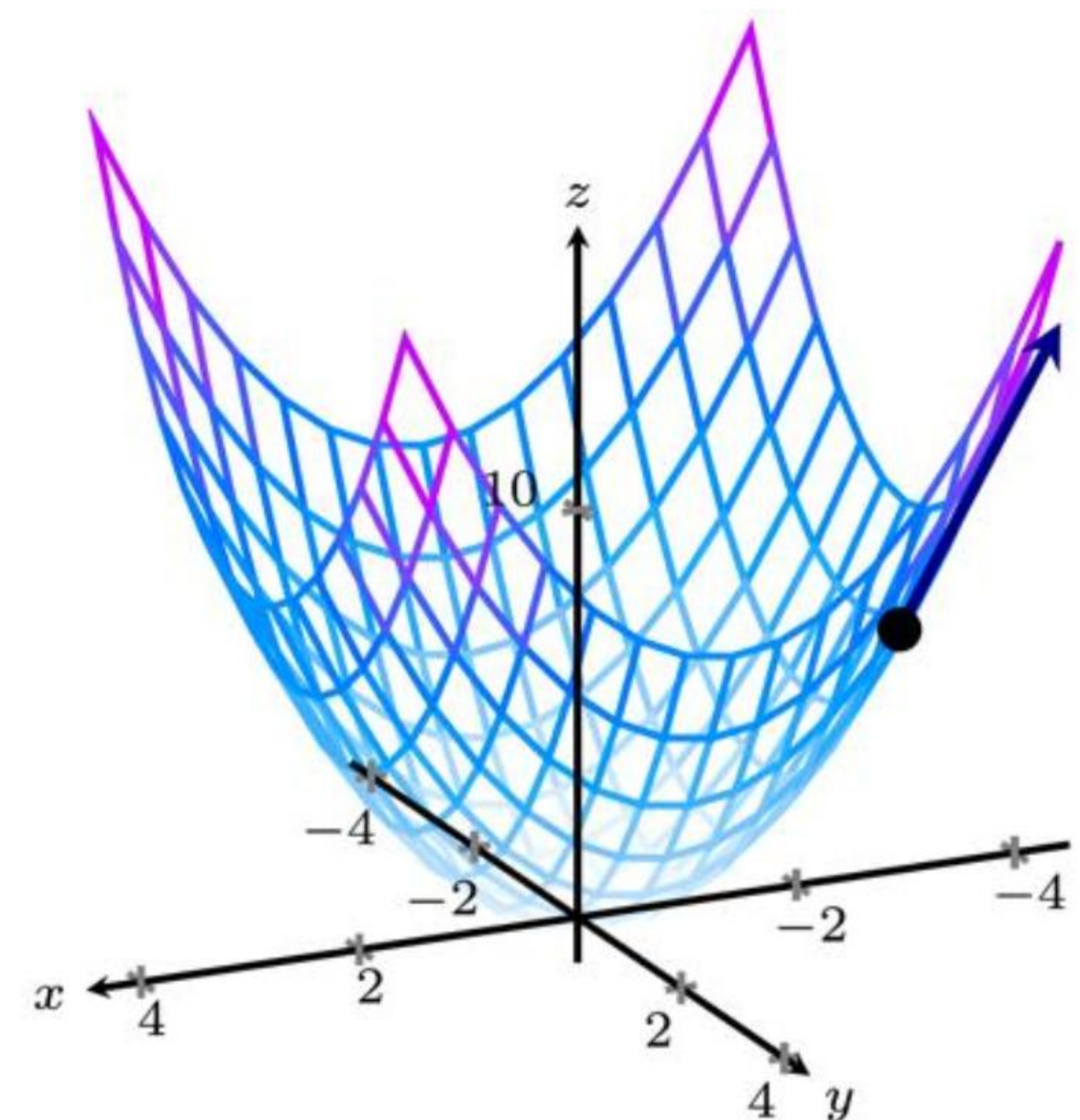
$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} J(\mathbf{w}_k)$

if converge then

return $\mathbf{w}_{k+1}$

end

end



Binomial Logistic Regression

➤ Learning (Binary)

$$\begin{aligned}\nabla_{\mathbf{w}} J(\mathbf{w}) &= \nabla_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N L_{BCE}(f_{\mathbf{w}}(\mathbf{x}_i), y_i) \\ &= \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{w}} L_{BCE}(f_{\mathbf{w}}(\mathbf{x}_i), y_i) \\ &= \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{w}} (-y_i \ln(f_{\mathbf{w}}(\mathbf{x}_i)) - (1 - y_i) \ln(1 - f_{\mathbf{w}}(\mathbf{x}_i))) \\ &= -\frac{1}{N} \sum_{i=1}^N (y_i \nabla_{\mathbf{w}} \ln(f_{\mathbf{w}}(\mathbf{x}_i)) + (1 - y_i) \nabla_{\mathbf{w}} \ln(1 - f_{\mathbf{w}}(\mathbf{x}_i)))\end{aligned}$$

By chain rule:

$$\frac{\partial y(u(x))}{\partial x} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial x}, \quad \frac{\frac{d \ln(f_{\mathbf{w}}(\mathbf{x}_i))}{d f_{\mathbf{w}}(\mathbf{x}_i)} \frac{d f_{\mathbf{w}}(\mathbf{x}_i)}{d \mathbf{w}}}{\frac{1}{f_{\mathbf{w}}(\mathbf{x}_i)} \nabla_{\mathbf{w}} f_{\mathbf{w}}(\mathbf{x}_i)} \quad - \frac{1}{1 - f_{\mathbf{w}}(\mathbf{x}_i)} \nabla_{\mathbf{w}} f_{\mathbf{w}}(\mathbf{x}_i)$$

$$= -\frac{1}{N} \sum_{i=1}^N \left(\frac{y_i}{f_{\mathbf{w}}(\mathbf{x}_i)} \nabla_{\mathbf{w}} f_{\mathbf{w}}(\mathbf{x}_i) - \frac{1 - y_i}{1 - f_{\mathbf{w}}(\mathbf{x}_i)} \nabla_{\mathbf{w}} f_{\mathbf{w}}(\mathbf{x}_i) \right)$$

Continue on next page...

Binomial Logistic Regression

➤ Learning (Binary)

$$\begin{aligned}\nabla_{\mathbf{w}} J(\mathbf{w}) &= \nabla_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N L_{BCE}(f_{\mathbf{w}}(\mathbf{x}_i), y_i) \\ &= -\frac{1}{N} \sum_{i=1}^N \left(\frac{y_i}{f_{\mathbf{w}}(\mathbf{x}_i)} \nabla_{\mathbf{w}} f_{\mathbf{w}}(\mathbf{x}_i) - \frac{1-y_i}{1-f_{\mathbf{w}}(\mathbf{x}_i)} \nabla_{\mathbf{w}} f_{\mathbf{w}}(\mathbf{x}_i) \right)\end{aligned}$$

$$\begin{aligned}\nabla_{\mathbf{w}} f_{\mathbf{w}}(\mathbf{x}_i) &= \nabla_{\mathbf{w}} \sigma(\mathbf{x}_i^T \mathbf{w}) \\ &= \sigma'(z_i) \mathbf{x}_i\end{aligned}$$

By chain rule and $\frac{d(\mathbf{b}^T \mathbf{w})}{d\mathbf{w}} = \mathbf{b}$

$$= \sigma(z_i)(1 - \sigma(z_i)) \mathbf{x}_i$$

$$= f_{\mathbf{w}}(\mathbf{x}_i)(1 - f_{\mathbf{w}}(\mathbf{x}_i)) \mathbf{x}_i$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N,1} & \dots & x_{N,d} \end{bmatrix}$$

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i (f_{\mathbf{w}}(\mathbf{x}_i) - y_i) = \frac{1}{N} \mathbf{X}^T (\mathbf{f}_{\mathbf{w}}(\mathbf{X}) - \mathbf{y})$$

Binomial Logistic Regression

Proof.
$$\begin{aligned}\frac{\partial \sigma(a)}{\partial a} &= \frac{\partial}{\partial a} \left(\frac{1}{1 + \exp(-\beta a)} \right) \\ &= -\frac{1}{(1 + e^{-\beta a})^2} \frac{\partial(1 + e^{-\beta a})}{\partial a} \\ &= \frac{\beta}{(1 + e^{-\beta a})^2} e^{-\beta a} \\ &= \frac{\beta}{(1 + e^{-\beta a})^2} (1 + e^{-\beta a} - 1) \\ &= \beta \left(\frac{1}{1 + e^{-\beta a}} - \frac{1}{(1 + e^{-\beta a})^2} \right) \\ &= \beta (\sigma(a) - \sigma^2(a)) \\ &= \beta \sigma(a)(1 - \sigma(a))\end{aligned}$$

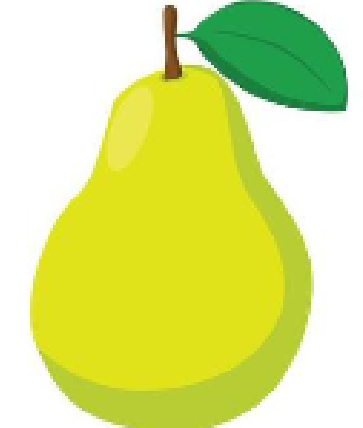
Take $\beta = 1$ for our case

Binomial Logistic Regression

➤ Prediction (Binary)

$$\mathbf{f}_{\mathbf{w}^*}(\mathbf{X}_t) = \sigma(\mathbf{X}_t \mathbf{w}^*) = \sigma \left(\begin{bmatrix} 1 & x_{1,1}^t & \dots & x_{1,d}^t \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1}^t & \dots & x_{m,d}^t \end{bmatrix} \begin{bmatrix} w_0^* \\ \vdots \\ w_d^* \end{bmatrix} \right) = \begin{bmatrix} \sigma((\mathbf{x}_1^t)^T \mathbf{w}^*) \\ \vdots \\ \sigma((\mathbf{x}_m^t)^T \mathbf{w}^*) \end{bmatrix}$$

Each entry is the probability of each test sample belonging to the binary label 1.

e.g.,  VS 
Label: 0 1

To predict class label,
set a threshold.

$$f_{\mathbf{w}}(\mathbf{x}_{new}) = 0.2$$

Probability of $\mathbf{x}_{new}$ belonging to
pear class (label 1) is 0.2

$$f_{\mathbf{w}}(\mathbf{x}_{new}) = 0.6$$

Probability of $\mathbf{x}_{new}$ belonging to
pear class (label 1) is 0.6

Binomial Logistic Regression

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class1}), (0.8, \text{class2})\}$, predict the probability of $\{x=-0.1\}$ belonging to *class 1* using binomial logistic regression.

Step 0: Prepare the training data

Binary encoding of the target output

class1: 1 *class2*: 0

$$\mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 0.5 \\ 1 & 0.3 \\ 1 & 0.8 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Binomial Logistic Regression

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class1}), (0.8, \text{class2})\}$, predict the probability of $\{x=-0.1\}$ belonging to class 1.

Step 1: Learning (work out $\mathbf{w}^*$)

Gradient Descent:

-Gradient $\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^T (\mathbf{f}_{\mathbf{w}}(\mathbf{X}) - \mathbf{y})$

-Initialize $\mathbf{w}_0 = \begin{bmatrix} 0.1 \\ -1 \end{bmatrix}$, learning rate $\eta = 0.1$

-At each iteration, $\mathbf{w}_{k+1} = \mathbf{w}_k - \eta \nabla_{\mathbf{w}} J(\mathbf{w}_k)$

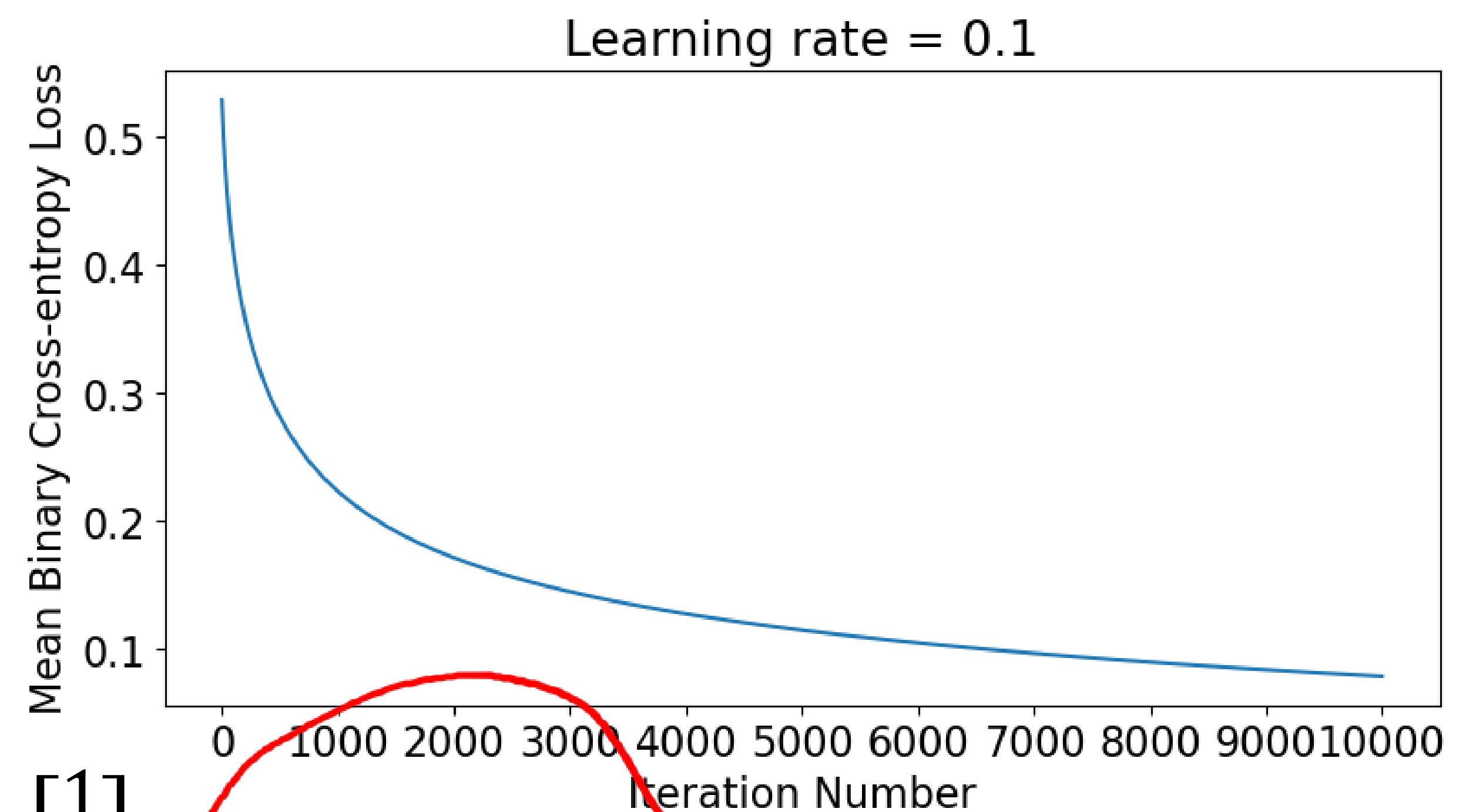
$\mathbf{w}_0 = \begin{bmatrix} 0.1 \\ -1 \end{bmatrix}$

$$\nabla_{\mathbf{w}} J(\mathbf{w}_0) = \frac{1}{5} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ -1 & 0 & 0.5 & 0.3 & 0.8 \end{bmatrix} \left(\begin{bmatrix} \frac{1}{1 + e^{-\mathbf{x}_1^T \mathbf{w}_0}} \\ \vdots \\ \frac{1}{1 + e^{-\mathbf{x}_5^T \mathbf{w}_0}} \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} -0.1083 \\ 0.1102 \end{bmatrix}$$

$\mathbf{w}_1 = \mathbf{w}_0 - \eta \nabla_{\mathbf{w}} J(\mathbf{w}_0) = \begin{bmatrix} 0.1108 \\ -1.0110 \end{bmatrix}$

$\vdots$

$\mathbf{w}_{10000} = \begin{bmatrix} 6.0786 \\ -15.3051 \end{bmatrix}$



Binomial Logistic Regression

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class1}), (0.8, \text{class2})\}$, predict the probability of $\{x=-0.1\}$ belonging to class 1.

Step 2: Prediction

Augment **1** for the feature vectors

$$\mathbf{x}_t = \begin{bmatrix} 1 \\ -0.1 \end{bmatrix}$$

$$\mathbf{w}_{10000} = \begin{bmatrix} 6.0786 \\ -15.3051 \end{bmatrix}$$

$$y_t^* = \sigma(\mathbf{x}_t^T \mathbf{w}^*)$$

$$= \sigma \left(\begin{bmatrix} 1 & -0.1 \end{bmatrix} \begin{bmatrix} 6.0786 \\ -15.3051 \end{bmatrix} \right)$$

$$= \frac{1}{1 + e^{-7.6091}}$$

$$= 0.9995$$

If the threshold is set to 0.5,

$$i.e., \sigma(z) = \frac{1}{1 + e^{-z}} \begin{cases} > 0.5, \text{class1} \\ < 0.5, \text{class2} \end{cases}$$

Then, ~~$\frac{1}{1 + e^{-(\mathbf{x}^T \mathbf{w}^*)}} = 0.5$~~ is the decision boundary.

$$\Downarrow$$
$$\mathbf{x}^T \mathbf{w}^* = 0$$

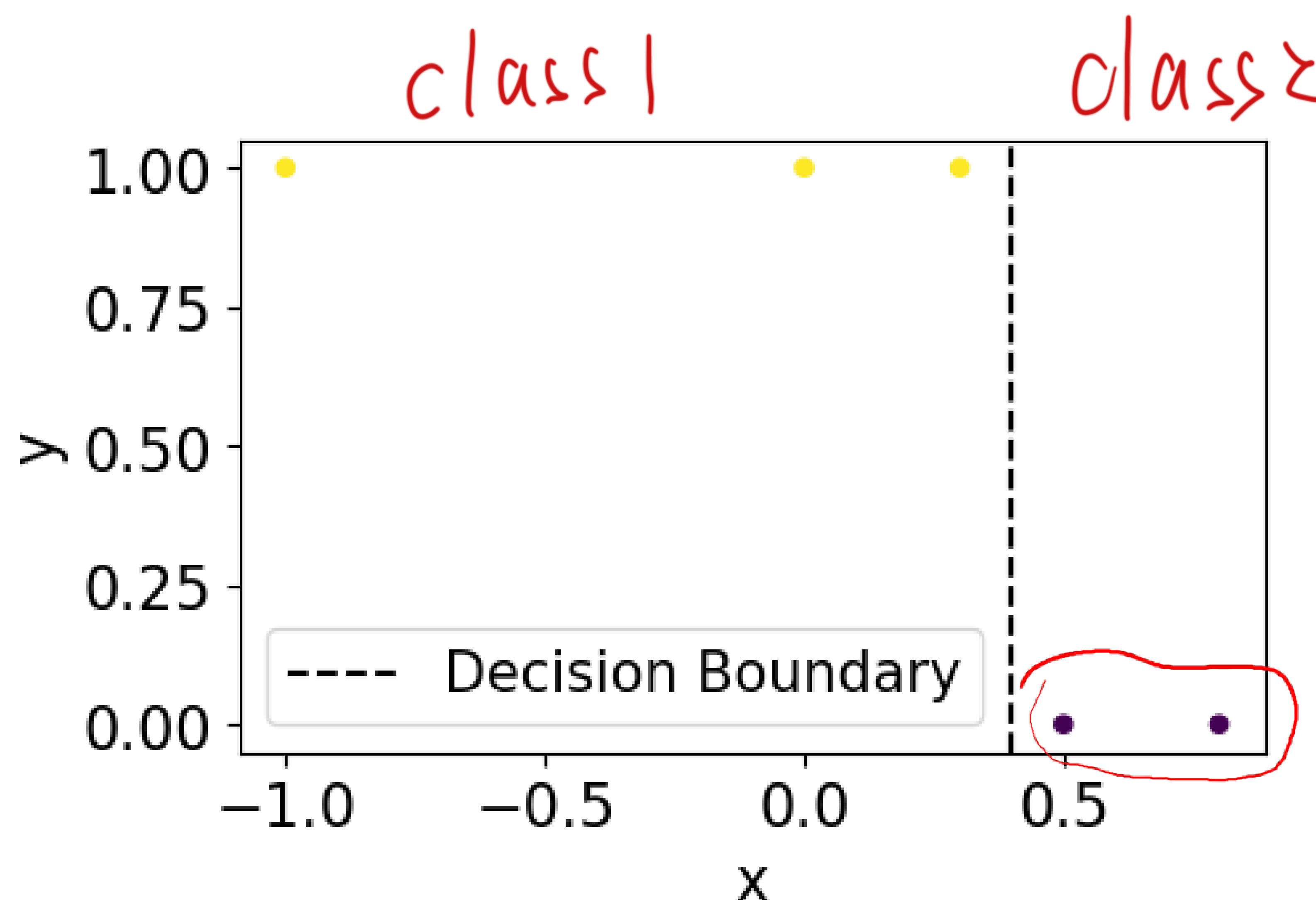
Boundary

$$\begin{bmatrix} 1 & x_1 \end{bmatrix} \begin{bmatrix} 6.0786 \\ -15.3051 \end{bmatrix} = 0$$

$$x_1 = 0.3971$$

Binomial Logistic Regression

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class1}), (0.8, \text{class2})\}$, predict the probability of $\{x=-0.1\}$ belonging to class 1.



If the threshold is set to 0.5,

$$i.e., \sigma(z) = \frac{1}{1 + e^{-z}} \begin{cases} > 0.5, \text{class1} \\ < 0.5, \text{class2} \end{cases}$$

Then, $\frac{1}{1 + e^{-(x^T \mathbf{w}^*)}} = 0.5$ is the decision boundary.

$$\mathbf{x}^T \mathbf{w}^* = 0$$

$$\begin{bmatrix} 1 & x_1 \end{bmatrix} \begin{bmatrix} 6.0786 \\ -15.3051 \end{bmatrix} = 0$$

$$x_1 = 0.3971$$

Multinomial Logistic Regression

➤ Model (Multi-class)

Input: $(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_N, \mathbf{y}_N),$

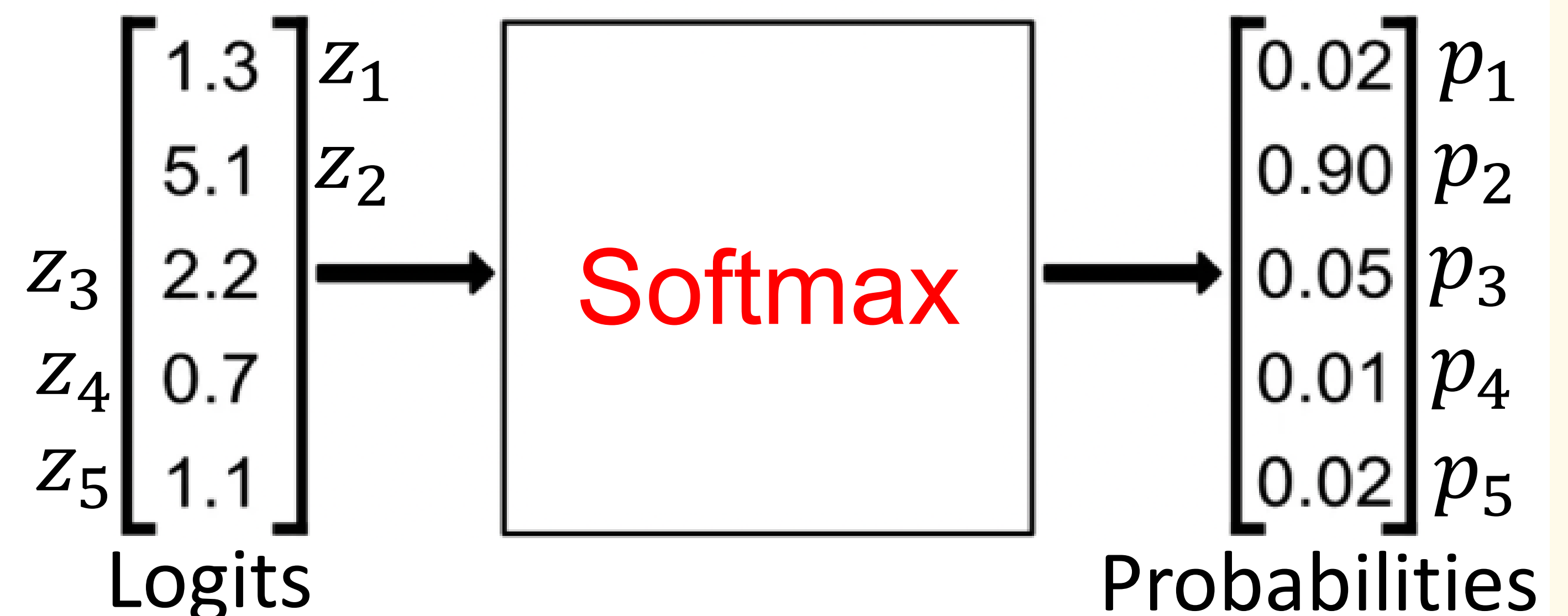
Target output $\mathbf{y}_i$ is one-hot encoded vector

$$\begin{aligned} \text{Output: } \mathbf{f}_{\mathbf{W}}(\mathbf{x}) &= \boldsymbol{\sigma}(\mathbf{x}^T \mathbf{W}) = \boldsymbol{\sigma}\left(\begin{bmatrix} 1 & x_1 & \cdots & x_d \end{bmatrix} \begin{bmatrix} \mathbf{w}_{0,1} & \cdots & \mathbf{w}_{0,K} \\ \mathbf{w}_{1,1} & \cdots & \mathbf{w}_{1,K} \\ \vdots & \ddots & \vdots \\ \mathbf{w}_{d,1} & \cdots & \mathbf{w}_{d,K} \end{bmatrix}\right) \\ &= \boldsymbol{\sigma}\left(\begin{bmatrix} \mathbf{x}^T \mathbf{w}_1 & \cdots & \mathbf{x}^T \mathbf{w}_K \end{bmatrix}\right) \\ &= \boldsymbol{\sigma}\left(\begin{bmatrix} z_1 & \cdots & z_K \end{bmatrix}\right) \\ &= \begin{bmatrix} \frac{e^{z_1}}{\sum_{j=1}^K e^{z_j}} & \cdots & \frac{e^{z_K}}{\sum_{j=1}^K e^{z_j}} \end{bmatrix} \end{aligned}$$

K classes

where $\boldsymbol{\sigma}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, k = 1, 2, \dots, K$

is **Softmax** Function



Multinomial Logistic Regression

➤ Learning (Multi-class)

Find the optimal values for $\mathbf{W}^*$ which minimizes:

$$J(\mathbf{W}) = \frac{1}{N} \sum_{i=1}^N L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)$$

where $L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i) = -\sum_{k=1}^{(K)} y_{i,k} \ln(p_{i,k})$.
Categorical cross-entropy loss

$p_{i,k}$: k -th element of output vector $\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i)$

$y_{i,k}$: k -th element of target vector $\mathbf{y}_i$

probability it belongs to class k

$$L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i) = -\ln(p_{i,c}), \text{ where } y_{i,c} = 1$$

Only the target/ground-truth class contributes

$$\mathbf{x}_i \text{ from class } c, \quad \mathbf{y}_i = [0, \dots, 0, 1, 0, \dots, 0]$$

$$\begin{array}{ccc} \downarrow & \downarrow & \downarrow \\ y_{i,1} & y_{i,c} & y_{i,K} \end{array}$$

Multinomial Logistic Regression

➤ Learning (Multi-class)

$$\operatorname{argmin}_{\mathbf{W}} \frac{1}{N} \sum_{i=1}^N L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i) \quad \text{no closed-form solution!}$$

Use Gradient Descent Algorithm!

$$\frac{dJ(\mathbf{W})}{d\mathbf{W}} = \frac{d}{d\mathbf{W}} \left(\frac{1}{N} \sum_{i=1}^N L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i) \right) = \frac{1}{N} \sum_{i=1}^N \frac{d}{d\mathbf{W}} L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)$$

$$= \frac{1}{N} \sum_{i=1}^N \begin{bmatrix} \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{0,1}} & \cdots & \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{0,K}} \\ \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{1,1}} & \cdots & \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{1,K}} \\ \vdots & \ddots & \vdots \\ \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{d,1}} & \cdots & \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{d,K}} \end{bmatrix}$$

$$\mathbf{W} = \begin{bmatrix} w_{0,1} & \cdots & w_{0,K} \\ w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{bmatrix}$$

Continue on next page...

Multinomial Logistic Regression

➤ Learning (Multi-class)

$$\nabla \frac{\partial L_{CCE}(\mathbf{f}_W(\mathbf{x}_i), y_i)}{\partial w_{q,g}} = - \frac{\partial \sum_{k=1}^K y_{i,k} \ln(p_{i,k})}{\partial w_{q,g}}$$

$$p_{i,k} = \frac{\sigma(\mathbf{z}_i)_k}{\sum_{j=1}^K e^{z_{i,j}}}$$

$$= - \sum_{k=1}^K y_{i,k} \frac{\partial \ln(p_{i,k})}{\partial w_{q,g}}$$

By chain rule

$$= - \sum_{k=1}^K \frac{y_{i,k}}{p_{i,k}} \frac{\partial}{\partial w_{q,g}} \left(\frac{e^{z_{i,k}}}{\sum_{j=1}^K e^{z_{i,j}}} \right)$$

By chain rule

$$= - \sum_{k=1}^K \frac{y_{i,k}}{p_{i,k}} \frac{\partial}{\partial z_{i,g}} \left(\frac{e^{z_{i,k}}}{\sum_{j=1}^K e^{z_{i,j}}} \right) \frac{\partial z_{i,g}}{\partial w_{q,g}}$$

Continue on next page...

Multinomial Logistic Regression

➤ Learning (Multi-class)

$$\begin{aligned}
 & \bullet \frac{\partial}{\partial z_{i,g}} \left(\frac{e^{z_{i,k}}}{\sum_{j=1}^K e^{z_{i,j}}} \right) \begin{cases} k = g: e^{z_{i,g}} \\ k \neq g: 0 \end{cases} \\
 &= \frac{\left(\sum_{j=1}^K e^{z_{i,j}} \right) \frac{\partial e^{z_{i,k}}}{\partial z_{i,g}} - e^{z_{i,k}} \frac{\partial \left(\sum_{j=1}^K e^{z_{i,j}} \right)}{\partial z_{i,g}}}{\left(\sum_{j=1}^K e^{z_{i,j}} \right)^2} \\
 &= \frac{\left(\sum_{j=1}^K e^{z_{i,j}} \right) \delta_{kg} e^{z_{i,g}} - e^{z_{i,k}} e^{z_{i,g}}}{\left(\sum_{j=1}^K e^{z_{i,j}} \right)^2} \\
 &= \frac{\delta_{kg} e^{z_{i,g}}}{\sum_{j=1}^K e^{z_{i,j}}} - \frac{e^{z_{i,k}}}{\sum_{j=1}^K e^{z_{i,j}}} \frac{e^{z_{i,g}}}{\sum_{j=1}^K e^{z_{i,j}}} \\
 &= \delta_{kg} p_{i,g} - p_{i,k} p_{i,g} \\
 &= (\delta_{kg} - p_{i,k}) p_{i,g}
 \end{aligned}$$

By quotient rule:

$$\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{g(x) f'(x) - f(x) g'(x)}{(g(x))^2}$$

$$\delta_{kg} = \begin{cases} 1, & \text{if } k = g \\ 0, & \text{if } k \neq g \end{cases}$$

Multinomial Logistic Regression

➤ Learning (Multi-class)

$$\blacklozenge \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{q,g}} = - \sum_{k=1}^K \frac{y_{i,k}}{p_{i,k}} \frac{\partial}{\partial z_{i,g}} \left(\frac{e^{z_{i,k}}}{\sum_{j=1}^K e^{z_{i,j}}} \right) \frac{\partial z_{i,g}}{\partial w_{q,g}}$$

$$\frac{\partial z_{i,g}}{\partial w_{q,g}} = \frac{\partial (\mathbf{x}_i^T \mathbf{w}_g)}{\partial w_{q,g}}$$

$$= \frac{\partial}{\partial w_{q,g}} (x_{i,0} w_{0,g} + \dots + x_{i,d} w_{d,g})$$

$$= x_{i,q}$$

$$\mathbf{W} = \begin{bmatrix} w_{0,1} & \dots & w_{0,K} \\ w_{1,1} & \dots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \dots & w_{d,K} \end{bmatrix}$$

$$= [\mathbf{w}_1 \quad \dots \quad \mathbf{w}_K]$$

Multinomial Logistic Regression

➤ Learning (Multi-class)

$$\blacklozenge \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{q,g}} = - \sum_{k=1}^K \frac{y_{i,k}}{p_{i,k}} \frac{\partial}{\partial z_{i,g}} \left(\frac{e^{z_{i,k}}}{\sum_{j=1}^K e^{z_{i,j}}} \right) \frac{\partial z_{i,g}}{\partial w_{q,g}}$$

$$= - \sum_{k=1}^K \frac{y_{i,k}}{p_{i,k}} (\delta_{kg} - p_{i,k}) p_{i,g} x_{i,q}$$

$$\delta_{kg} = \begin{cases} 1, & \text{if } k = g \\ 0, & \text{if } k \neq g \end{cases}$$

$$= p_{i,g} x_{i,q} \sum_{k=1}^K \frac{y_{i,k}}{p_{i,k}} (p_{i,k} - \delta_{kg})$$

$$= p_{i,g} x_{i,q} \left(\sum_{k=1}^K y_{i,k} - \frac{y_{i,g}}{p_{i,g}} \right)$$

$$= p_{i,g} x_{i,q} \left(1 - \frac{y_{i,g}}{p_{i,g}} \right)$$

$$= x_{i,q} (p_{i,g} - y_{i,g})$$

Multinomial Logistic Regression

➤ Learning (Multi-class)

$$\frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{q,g}} = x_{i,q}(p_{i,g} - y_{i,g})$$

$$\diamond \frac{dJ(\mathbf{W})}{d\mathbf{W}} = \frac{1}{N} \sum_{i=1}^N \frac{d}{d\mathbf{W}} L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)$$

$$= \frac{1}{N} \sum_{i=1}^N \begin{bmatrix} \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{0,1}} & \cdots & \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{0,K}} \\ \vdots & \ddots & \vdots \\ \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{d,1}} & \cdots & \frac{\partial L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)}{\partial w_{d,K}} \end{bmatrix}$$

$$= \frac{1}{N} \sum_{i=1}^N \begin{bmatrix} x_{i,0}(p_{i,1} - y_{i,1}) & \cdots & x_{i,0}(p_{i,K} - y_{i,K}) \\ \vdots & \ddots & \vdots \\ x_{i,d}(p_{i,1} - y_{i,1}) & \cdots & x_{i,d}(p_{i,K} - y_{i,K}) \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & \cdots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N,1} & \cdots & x_{N,d} \end{bmatrix}$$

$\mathbf{Y}$ has a size of $N \times K$

$$= \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i (\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i) - \mathbf{y}_i) = \frac{1}{N} \mathbf{X}^T (\mathbf{F}_{\mathbf{W}}(\mathbf{X}) - \mathbf{Y})$$

Exactly the same as binomial Logistic Regression! Why?

Multinomial Logistic Regression

- $\text{Softmax}() = \text{Logistic}()$, when $K=2$

Suppose class 1 is assigned with label 1

$$\begin{aligned}\sigma(\mathbf{z})_1 &= \frac{e^{z_1}}{e^{z_1} + e^{z_2}} \\ &= \frac{1}{1 + e^{z_2 - z_1}}\end{aligned}$$

$$\begin{aligned}\sigma(z) &= \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-\mathbf{x}^T \mathbf{w}}} \\ &= P(y = 1 | \mathbf{x})\end{aligned}$$

$$\text{Let } a = z_1 - z_2 = \mathbf{x}^T \underbrace{(\mathbf{w}_1 - \mathbf{w}_2)}_{\mathbf{w}}$$

$$\begin{aligned}\sigma(\mathbf{z})_1 &= \frac{1}{1 + e^{-a}} = \frac{1}{1 + e^{-\mathbf{x}^T \mathbf{w}}} \\ &= P(y = 1 | \mathbf{x})\end{aligned}$$

Multinomial Logistic Regression

- $L_{CCE} = L_{BCE}$, when $K=2$

Suppose class 1 is assigned with label 1

$$\underbrace{L_{CCE}}_{\text{Softmax Function}} = - \sum_{k=1}^2 y_k \ln(p_k) \qquad L_{BCE} = -y \ln(p) - (1-y) \ln(1-p)$$
$$= -y_1 \ln(p_1) - y_2 \ln(p_2)$$
$$= -y_1 \ln(p_1) - (1-y_1) \ln(1-p_1)$$

Softmax Function

Multinomial Logistic Regression

➤ Prediction (Multi-class)

$$\mathbf{F}_{\mathbf{W}^*}(\mathbf{X}_t) = \boldsymbol{\sigma}(\mathbf{X}_t \mathbf{W}^*) = \boldsymbol{\sigma} \left(\begin{bmatrix} \mathbf{1} & x_{1,1}^t & \dots & x_{1,d}^t \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{1} & x_{m,1}^t & \dots & x_{m,d}^t \end{bmatrix} \begin{bmatrix} w_{0,1}^* & \dots & w_{0,K}^* \\ w_{1,1}^* & \dots & w_{1,K}^* \\ \vdots & \ddots & \vdots \\ w_{d,1}^* & \dots & w_{d,K}^* \end{bmatrix} \right)$$

$$= \begin{bmatrix} \boldsymbol{\sigma}((\mathbf{x}_1^t)^T \mathbf{W}^*) \\ \vdots \\ \boldsymbol{\sigma}((\mathbf{x}_m^t)^T \mathbf{W}^*) \end{bmatrix}$$

Each row is the probability distribution for each test sample

e.g., 2 test samples, 4 classes

$$\begin{bmatrix} 0.1 & 0.7 & 0 & 0.2 \\ 0.3 & 0.3 & 0.4 & 0 \end{bmatrix}$$

To predict the class label, output column index that has the largest probability for each test sample.

Multinomial Logistic Regression

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class3}), (0.8, \text{class2})\}$, predict the probability of $\{x=-0.1\}$ belonging to each class using multinomial logistic regression.

Multinomial Logistic Regression

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class3}), (0.8, \text{class2})\}$, predict the probability of $\{x=-0.1\}$ belonging to each class using multinomial logistic regression.

Step 0: Prepare the training data

One-hot encoding of the target output

class1: [1,0,0] *class2*: [0,1,0] *class3*: [0,0,1]

$$\mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 0.5 \\ 1 & 0.3 \\ 1 & 0.8 \end{bmatrix}$$

$$\mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

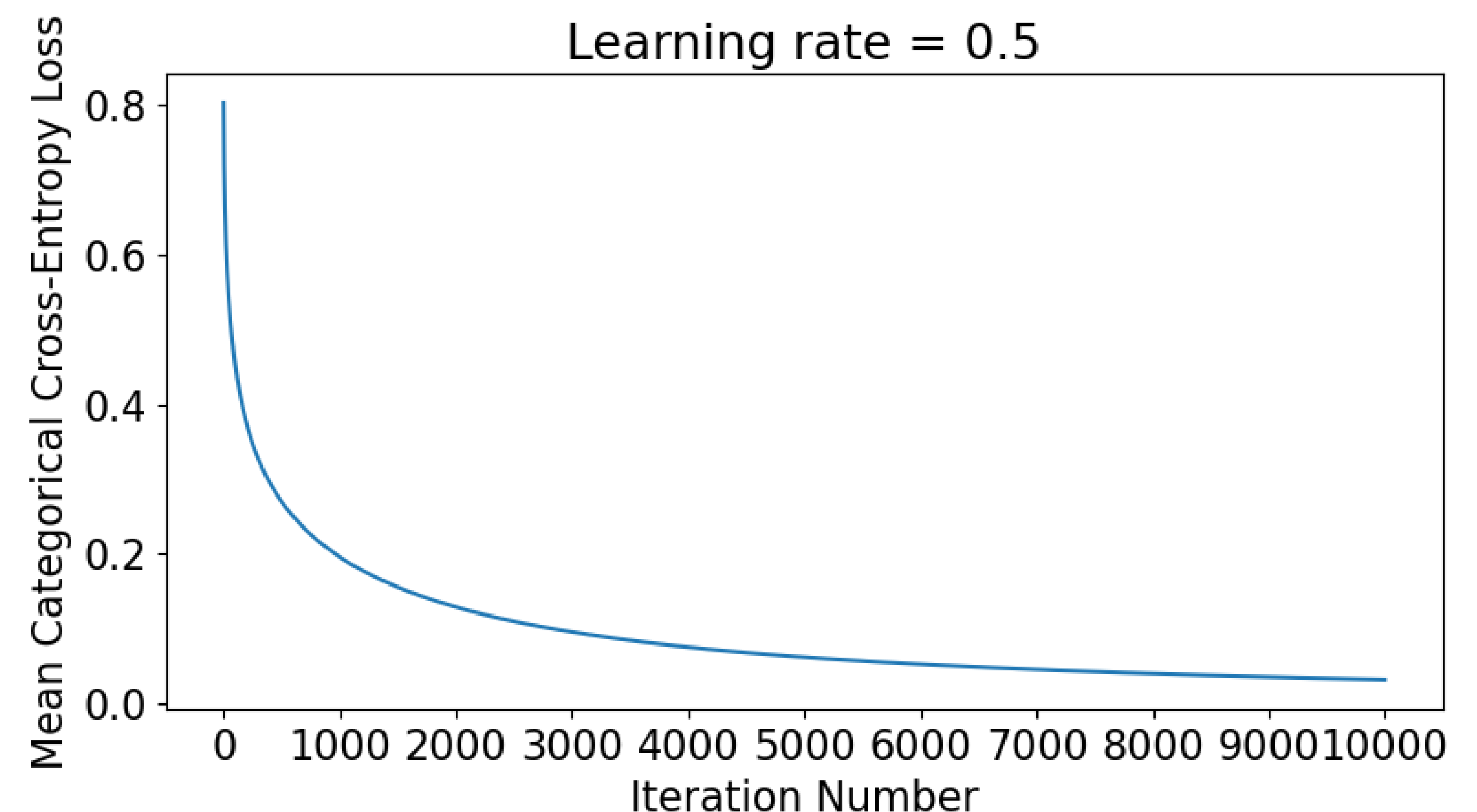
Multinomial Logistic Regression

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class3}), (0.8, \text{class2})\}$, predict the probability of $\{x=-0.1\}$ belonging to each class using multinomial logistic regression.

Step 1: Learning (work out $\mathbf{W}^*$)

Gradient Descent:

$$\mathbf{W}_{10000} = \begin{bmatrix} 6.5540 & -8.5683 & 3.0143 \\ -25.6024 & 27.1366 & -1.9341 \end{bmatrix}$$



-Gradient $\nabla_{\mathbf{W}} J(\mathbf{W}) = \frac{1}{N} \mathbf{X}^T (\mathbf{F}_{\mathbf{W}}(\mathbf{X}) - \mathbf{Y})$

-Initialize $\mathbf{W}_0 = \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ -1 & 0.5 & 0.1 \end{bmatrix}$,

learning rate $\eta = 0.5$

-At each iteration, $\mathbf{W}_{k+1} = \mathbf{W}_k - \eta \nabla_{\mathbf{W}} J(\mathbf{W}_k)$

$$\mathbf{W}_0 = \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ -1 & 0.5 & 0.1 \end{bmatrix}$$

$$\nabla_{\mathbf{W}} J(\mathbf{W}_0) = \frac{1}{N} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ -1 & 0 & 0.5 & 0.3 & 0.8 \end{bmatrix} \left(\begin{bmatrix} \text{Softmax}(\mathbf{x}_1^T \mathbf{W}_0) \\ \vdots \\ \text{Softmax}(\mathbf{x}_5^T \mathbf{W}_0) \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \right)$$

$$\vdots = \begin{bmatrix} -0.0355 & -0.0479 & 0.0834 \\ 0.1321 & -0.1402 & 0.0080 \end{bmatrix}$$

Multinomial Logistic Regression

Example: Given a training set $\{(x_i, y_i)\}_{i=1}^5 = \{(-1, \text{class1}), (0, \text{class1}), (0.5, \text{class2}), (0.3, \text{class3}), (0.8, \text{class2})\}$, predict the probability of $\{x=-0.1\}$ belonging to each class using multinomial logistic regression.

Step 2: Prediction

Augment **1** for the feature vector

$$\mathbf{x}_t = \begin{bmatrix} \mathbf{1} \\ -0.1 \end{bmatrix} \quad \mathbf{W}_{10000} = \begin{bmatrix} 6.5540 & -8.5683 & 3.0143 \\ -25.6024 & 27.1366 & -1.9341 \end{bmatrix}$$

$$y_t^* = \sigma(\mathbf{x}_t^T \mathbf{W}^*)$$

$$= \sigma \left([1 \quad -0.1] \begin{bmatrix} 6.5540 & -8.5683 & 3.0143 \\ -25.6024 & 27.1366 & -1.9341 \end{bmatrix} \right)$$

$$= \left[\frac{e^{9.1143}}{e^{9.1143} + e^{-11.2820} + e^{3.2077}}, \frac{e^{-11.2820}}{e^{9.1143} + e^{-11.2820} + e^{3.2077}}, \frac{e^{3.2077}}{e^{9.1143} + e^{-11.2820} + e^{3.2077}} \right]$$

$$= [9.9729 \times 10^{-1}, 1.3830 \times 10^{-9}, 2.7142 \times 10^{-3}]$$

Evaluation Metrics for Classification

Evaluation Metrics for Classification

➤ Binary Classification

Class1 (label +1 or 1): Positive Class

Class2 (label -1 or 0): Negative Class

TP: True Positive

FN: False Negative
(i.e., Type II Error)

FP: False Positive
(i.e., Type I Error)

TN: True Negative

Confusion Matrix for Binary Classification

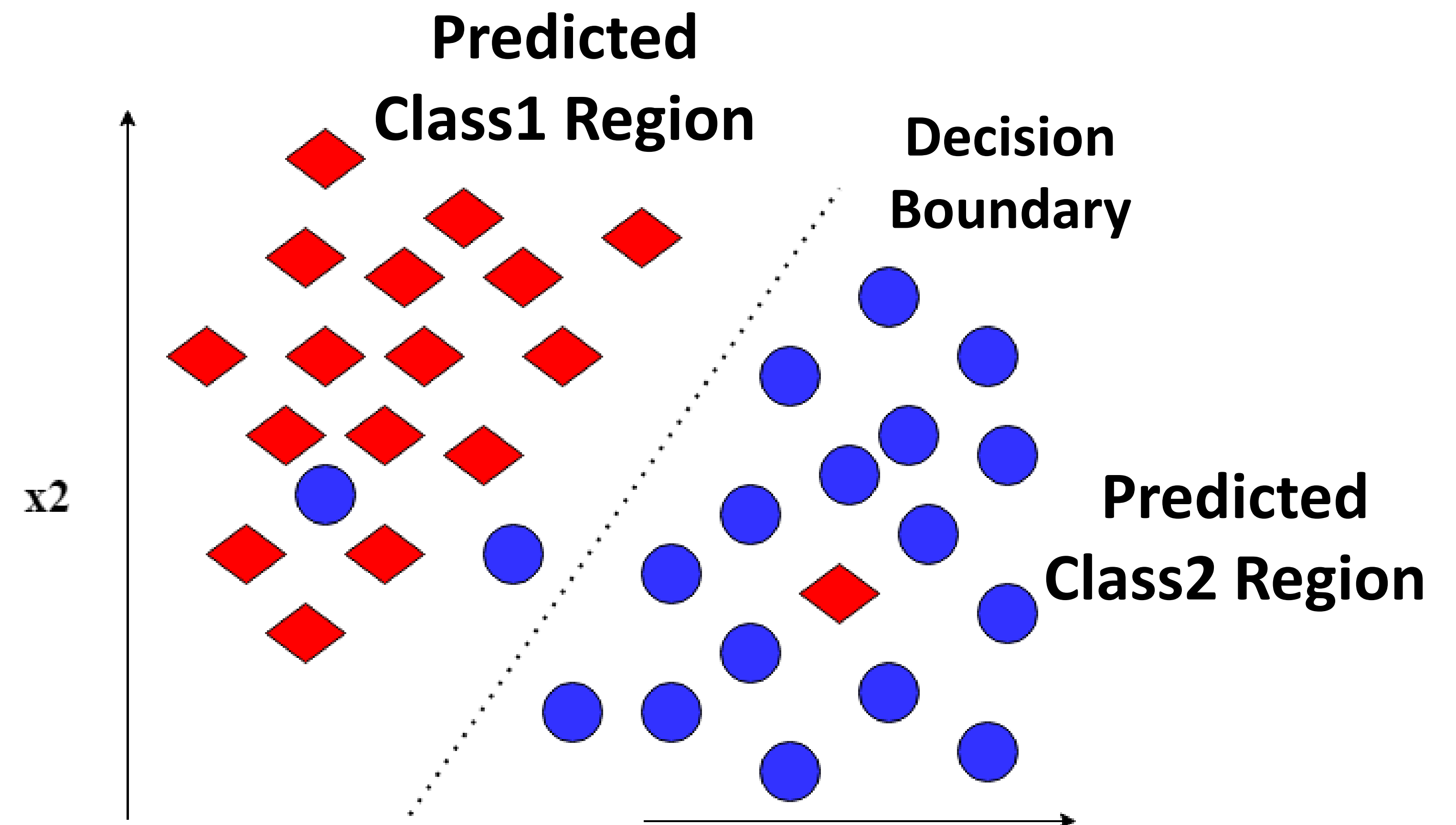
	$\hat{P}$ (predicted)	$\hat{N}$ (predicted)	
P (actual)	<i>Correctly predicted as positive</i> TP	<i>Wrongly predicted as negative</i> FN	Recall $TP/(TP+FN)$
N (actual)	FP	TN	
	Precision $TP/(TP+FP)$		Accuracy $(TP+TN)/(TP+TN+FP+FN)$

Evaluation Metrics for Classification

➤ Binary Classification

e.g.,

- ◆ Class1: Positive Class
- Class2: Negative Class



Confusion Matrix for Binary Classification

	$\hat{P}$ (predicted)	$\hat{N}$ (predicted)
P (actual)	16 TP	1 FN
N (actual)	2 FP	16 TN

Precision
 $TP/(TP+FP)$

Recall
 $TP/(TP+FN)$

Accuracy
 $(TP+TN)/(TP+TN+FP+FN)$

TP: True Positive
FN: False Negative
 (i.e., Type II Error)
FP: False Positive
 (i.e., Type I Error)
TN: True Negative

Evaluation Metrics for Classification

➤ Multicategory Classification

Confusion Matrix for Multicategory Classification

	$P_{\hat{1}}$ (predicted)	$P_{\hat{2}}$ (predicted)		$P_{\hat{C}}$ (predicted)
P_1 (actual)	$P_{1,\hat{1}}$	$P_{1,\hat{2}}$	...	$P_{1,\hat{C}}$
P_2 (actual)	$P_{2,\hat{1}}$	$P_{2,\hat{2}}$	...	$P_{2,\hat{C}}$
⋮	⋮	⋮	⋮	⋮
P_C (actual)	$P_{C,\hat{1}}$	$P_{C,\hat{2}}$		$P_{C,\hat{C}}$

```
from sklearn.metrics import confusion_matrix, accuracy_score  
cm = confusion_matrix(y_true, y_pred)  
acc = accuracy_score(y_true, y_pred)
```


Summary

✓ Polynomial Regression for Classification

Binary Classification

-Learning:

$$\mathbf{w}^* = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{y} \quad y_i \in \{-1, +1\}$$

-Prediction:

$$\mathbf{f}_{\mathbf{w}^*}(\mathbf{P}(\mathbf{X}_t)) = \text{sign}(\mathbf{P}_t \mathbf{w}^*)$$

Multi-category Classification

-Learning:

$$\mathbf{W}^* = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{Y}$$

$$\text{-Prediction: } \mathbf{F}_{\mathbf{W}^*}(\mathbf{P}(\mathbf{X}_t)) = \begin{bmatrix} \underset{k=1, \dots, K}{\text{argmax}}((\mathbf{p}_1^t)^T \mathbf{W}^*) \\ \vdots \\ \underset{k=1, \dots, K}{\text{argmax}}((\mathbf{p}_m^t)^T \mathbf{W}^*) \end{bmatrix}$$

✓ Logistic Regression

Binary Classification

-Learning: Gradient Descent

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^T (\mathbf{f}_{\mathbf{w}}(\mathbf{X}) - \mathbf{y})$$

-Prediction:

$$\mathbf{f}_{\mathbf{w}^*}(\mathbf{X}_t) = \sigma(\mathbf{X}_t \mathbf{w}^*) = \begin{bmatrix} \sigma((\mathbf{x}_1^t)^T \mathbf{w}^*) \\ \vdots \\ \sigma((\mathbf{x}_m^t)^T \mathbf{w}^*) \end{bmatrix}$$

Multi-category Classification

-Learning: Gradient Descent

$$\nabla_{\mathbf{W}} J(\mathbf{W}) = \frac{1}{N} \mathbf{X}^T (\mathbf{F}_{\mathbf{W}}(\mathbf{X}) - \mathbf{Y})$$

-Prediction:

$$\mathbf{F}_{\mathbf{W}^*}(\mathbf{X}_t) = \sigma(\mathbf{X}_t \mathbf{W}^*) = \begin{bmatrix} \sigma((\mathbf{x}_1^t)^T \mathbf{W}^*) \\ \vdots \\ \sigma((\mathbf{x}_m^t)^T \mathbf{W}^*) \end{bmatrix}$$

✓ Evaluation Metrics for Classification Task